Lab 2 | Data Wrangling in R

ST 437 Data Visualization

Student Name

March 26, 2026

Learning Objectives

1. **Understand the Importance of Data Wrangling**: Recognize the critical role of data wrangling in preparing datasets for accurate and meaningful analysis.
2. **Master Data Transformation Techniques**: Develop the ability to convert raw, untidy data into a structured format suitable for analysis and visualization.
3. **Emphasize Data Cleaning and Preparation**: Learn to identify and correct common data issues such as missing values, inconsistent formats, and incorrect entries.
4. **Ensure Data Accuracy and Consistency**: Gain skills in validating and maintaining the integrity of data throughout the wrangling process.
5. **Draw Reliable Conclusions**: Build confidence in making well-supported decisions and insights based on meticulously prepared and clean data.

Getting Started

First, ensure you have the necessary packages installed and loaded. We will use the `dplyr`, `lubridate`, `readr`, `ggplot2`, and `tidyr` packages for our examples.

> **!** Downloading R-packages
>
> Use `install.packages('Name of Package')` to install an R package. Careful! Package names are case sensitive, so install.packages('GGplot2') will not work, but `install.packages('ggplot2')` will.

```
library(ggplot2)
library(dplyr)
library(lubridate)
library(tidyr)
library(readr)
```

Download the Data

> Running Code Block Shortcuts
>
> `Mac users`: Use + return to run single or highlighted line(s). Use shift + return to run entire code block
> `Windows users`: Use ctrl + enter to run single or highlighted line(s). Use shift + enter to run entire code block

In the following code chunk, the `read_csv()` function is used to import the two required datasets for this lab. For this the chunk to run properly, ensure you have downloaded datasets from the Lab 2 Canvas page **and stored those datasets in the same file directory as this .qmd file**.

```
# Read the music data into R
spotify <- read_csv("/Users/madeleinesherry/Documents/st437/spotify.csv", show_col_types = FA
artists <- read_csv("/Users/madeleinesherry/Documents/st437/artists.csv", show_col_types = FA
```

Verify Datasets with `head()`

```
head(spotify, 5)
head(artists, 5)
```

Both datasets were found from Kaggle (https://www.kaggle.com/datasets/rishabhpancholi1302/spotify-most-popular-songs-dataset?resource=download and https://www.kaggle.com/datasets/vatsalmavani/spotify-dataset/data).

Description of the Main Dataset (`spotify.csv`)

Audio Features (Extracted from Spotify API): - **`danceability`** – How suitable a track is for dancing (0.0 – 1.0). - **`energy`** – Intensity and activity level of a song (0.0 – 1.0). - **`loudness`** – Overall loudness in decibels (dB). - **`speechiness`** – Presence of spoken words in the track (0.0 – 1.0). - **`acousticness`** – Probability that a track is acoustic (0.0 – 1.0). - **`instrumentalness`** – Predicts if a track is instrumental (0.0 – 1.0). - **`liveness`** – Probability of a live audience (0.0 – 1.0). - **`valence`** – Musical positivity or happiness (0.0 – 1.0). - **`tempo`** – Beats per minute (BPM) of the track. - **`key & mode`** – Musical key and mode (major/minor).

General Song Information: - **`song`** – Name of the song. - **`artist`** – Performing artist(s). - **`duration_ms`** – Duration of the song (in milliseconds). - **`release_date``** – Year when the song was released. - **genre** - Song's primary genre classification. - **popularity** - Spotify popularity metric (0 - 1). - **spotify_url`** – URL to the song on Spotify (not useful for us, but fun if you want to listen to the song!).

Additional Dataset (`artists.csv`)

- **`count`** - Number of songs released by the artist that are used to calculate the averages.
- **`artist`** - Performing artist(s).
- **`avg_danceability`** – Average over all the artist's songs of how suitable a track is for dancing (0.0 – 1.0).
- **`avg_duration_ms`** – Average duration of the artist's songs (in milliseconds)

Tidy Data Wrangling

Important Concepts

- `Joining`: Combine the main dataset with the additional data based on artist.
- `Filtering`: Filter data based on conditions such as year, artist, or genre.
- `Selecting`: Select specific columns for focused analysis.
- `Mutating`: Create new columns, such as the ratio between danceability and energy.
- `Summarizing`: Aggregate data by year, artist, or genre to find totals and averages.

Joining

Joining combines data from the main dataset with additional data.

Example

```r
joinedData <- left_join(spotify, artists, by = "artist")

head(joinedData, 5)
```

Visualization

```r
ggplot(joinedData, aes(x = genre, y = avg_danceability)) +
  geom_boxplot() +
  labs(title = "Average Danceability by Genre",
       x = "Genre", y = "Average Danceability") +
  theme_minimal()
```

Handling Dates

In some cases, it might be necessary to convert a `date` column from a categorical format into a proper date format for time series analysis or plotting purposes. Here's how you can do that in R using the `lubridate` package (Cheat Sheet here: https://rawgit.com/rstudio/cheatsheets/main/lubridate.pdf)

Though we use the `mutate` function before the section dedicated to it in this lab, it is necessary to put this first so that we can visualize trends over time throughout the rest of this lab.

Example

Converting 'release_date" to a date

Though it looks like our data is already in the correct format, the `release_date` column is actually in character format. Let's convert the `year` column to a date format. The `lubridate` package is handy for converting characters to dates. Because the variable is currently in the form "month/day/year", we'll use the `mdy()` function.

Note, there are 19 observations in this dataset whose release date was not formatted properly (e.g., only year was provided, a range of years were provided, etc.). Once the code chunk below is run, those observations will have `NA` listed for `release_date`.

```
# Ensure the lubridate package is loaded
joinedData <- joinedData |>
  mutate(release_date = ymd(release_date))

# Check the first few rows to see the new column
head(joinedData, 5)
```

Extracting the year from the `date` column.

Now, say we want to look at just the year that the song was released rather than the whole date. We can do that using the `year` function of the `lubridate` package.

```
# Extract just the year from the 'date' column and make a new column
joinedData <- joinedData |>
  mutate(year = year(release_date))
```

Visualization

```r
ggplot(joinedData, aes(x = year, y = loudness)) +
  geom_line() +
  labs(title = "Loudness Trends Over Time",
       x = "Year", y = "Loudness") +
  theme_minimal()
```

Challenges

1. **Create a new 'month' column from the 'date' column**:
    - **Task**: Extract the month from the `date`, and call it `month`.
    - **Purpose**: Useful for representing data that summarizes monthly results.

```r
# Your code goes here...
```

Filtering

Filtering is essential for narrowing down datasets to the most relevant information, making patterns easier to identify.

Example 1

You are tasked with visualizing trends in energy in Pop music released between 2015 and 2017. Notice: We are going to be using our adjusted dates that we created in the previous section.

Filtering Process

```r
filteredData1 <- joinedData |>
  filter(year >= 2015 & year <= 2017 & genre == "Pop")

head(filteredData1, 5)
```

Visualization

```r
ggplot(filteredData1, aes(x = release_date, y = energy, color = genre)) +
  geom_line() +
  labs(title = "Energy in Pop Music Trends (2015-2017)",
       x = "Date", y = "Energy") +
  theme_minimal()
```

What did we do here?

By focusing on specific genres and years, filtering allows for more targeted and relevant visualizations, making it easier to analyze trends and patterns specific to the context.

Example 2

Analyze the relationship between duration and speechiness in Rap music with energy less than 0.5 for the years 2015-2020.

Filtering Process

```
filteredData2 <- joinedData |>
  filter(year >= 2015 & year <= 2020 & genre == "Rap" & energy < 0.5)

head(filteredData2, 5)
```

Visualization

```
ggplot(filteredData2, aes(x = duration_ms, y = speechiness, color = energy)) +
  geom_point() +
  labs(title = "Song Duration vs. Speechiness (in Rap Songs with Low Energy 2015-2020)",
       x = "Song Duration (ms)", y = "Speechiness") +
  theme_minimal()
```

What did we do here?

Filtering based on song descriptors (energy, genre, and release date here) allows for focused analysis on the relationship between speechiness and duration, removing noise from different types of songs.

Filtering Challenges

1. **Recent Prolific Artists**:
    - **Filter**: Only songs released after 2020 and where the artist has released more than 25 songs (*Reminder*: the number of songs an artist has released comes from the 'count' variable we added).
    - **Purpose**: Isolate data for artists who have made a lot of music and have released songs recently.

```
# Your code goes here...
```

2. **Popular Rock Music**:
    - **Filter**: Show data only for the years 1975-1990 for Rock music with popularity over 0.3.
    - **Purpose**: Focus on popular Rock music from an era where it was popular.

```
# Your code goes here...
```

3. **Bruno's Most Danceable Songs**:
    - **Filter**: Data for Bruno Mars where the danceability is above 0.7.
    - **Purpose**: Analyze one artist's music with high danceability.

```
# Your code goes here...
```

Selecting

Selecting allows you to focus on specific columns relevant to your analysis.

Example

```
selectedData <- joinedData |>
  select(artist, genre, acousticness, loudness)

head(selectedData, 5)
```

Visualization

```
ggplot(selectedData, aes(x = acousticness, y = loudness)) +
  geom_smooth() +
  labs(title = "Acousticness vs. Loudness in Songs",
       x = "Acousticness", y = "Loudness") +
  theme_minimal()
```

Selecting Challenges

1. **Select columns** related to the overall 'upbeat-ness' of the songs (e.g., `danceability`, `energy`, `tempo`) for further analysis.

```
# Your code goes here...
```

2. **Create a dataset** with only the `liveness`, `valence`, and `mode` columns for all songs and show the first 5 rows.

```
# Your code goes here...
```

3. **Select and rename** the `duration_ms` and `avg_duration_ms` columns to `length` and `avg_length`, respectively.

*Hint check out the help documentation for the `?rename` function from the `dplyr` package.

```
# Your code goes here...
```

Mutating

Mutating helps create new columns based on existing data.

Example

```
mutatedData <- joinedData |>
  mutate(rel_danceability = danceability / avg_danceability)

head(mutatedData, 5)
```

Visualization

```
ggplot(mutatedData, aes(x = valence, y = rel_danceability, color = genre)) +
  geom_point(size = 2) +
  labs(title = "Valence vs. Relative Danceability",
       x = "Valence", y = "Relative Danceability") +
  geom_hline(yintercept = 1)+
  theme_minimal()
```

Mutating Challenges

1. **Add a new column** to `mutatedData` that indicates whether a song's tempo is above or below a certain threshold (e.g., 0.6).

```
# Your code goes here...
```

2. **Mutate the `duration_ms` column** to create a new column, `duration_mins`, that converts milliseconds to minutes.

Hint: Minutes = Milliseconds/60,000

```
# Your code goes here...
```

3. **Generate a column** that calculates the ratio of acousticness to liveness for each song.

```
# Your code goes here...
```

Summarizing

Summarizing aggregates data by some variable to find totals and averages.

Example

```
summaryData <- joinedData |>
  group_by(genre) |>
  summarize(
    avg_danceability = mean(danceability, na.rm = TRUE)
  )

head(summaryData, 5)
```

Visualization

```
ggplot(summaryData, aes(x = genre, y = avg_danceability, fill = genre)) +
  geom_bar(stat='identity')+
  labs(title = "Genre vs. Average Song Danceability",
       x = "Genre", y = "Average Song Danceability") +
  theme_minimal()
```

Summarizing Challenges

1. **Summarize the dataset** by finding the average popularity for all songs.

```
# Your code goes here...
```

2. **For each artist,** find the total milliseconds of songs released and average tempo.

```
# Your code goes here...
```

3. **Group the data** by genre and summarize to find the most popular song per genre.

```
# Your code goes here...
```

4. **Summarize by genre** to find the total duration per genre and average valence per genre.

```
# Your code goes here...
```

Untidy Data Wrangling

Handling Wide vs. Long Formats

Untidy data often comes in a "wide" format, where multiple variables are stored across columns rather than in a long format where each observation is a row.

Creating an Untidy Version

To demonstrate this, let's first work with a very small dataset. We'll create that dataset below.

```
temps_wide <- tribble(
  ~ day, ~ air, ~ soil,
  1, 27, 15,
  2, 31, 16,
  3, 22, 15
)

temps_wide
```

To pivot the `temps_wide` dataset so that is in a tidy format, we can use the `pivot_longer()` function. This function takes the data as the first argument (below we'll specify that with a pipe) and the columns you want to stack in the second argument.

```
temps_long <- temps_wide |>
  pivot_longer(c(air, soil))

temps_long
```

If you want to create more specific column headers in the long format, you can do using the `names_to` and `values_to arguments`.

```
temps_long <- temps_wide |>
  pivot_longer(c(air, soil), names_to = "type", values_to = "temp")

temps_long
```

We can do the inverse and widen a data set.

To demonstrate this, let's take our dataset and select only the `song`, `year`, and `popularity`.

```
# Create a wide format from the existing long format
popData <- joinedData |>
  select(song, year, popularity)
```

With the three column dataset, convert it into a wide format, then back to a long format.

```
wideData <- joinedData |>
  pivot_wider(names_from = song,
              values_from = popularity,
              id_cols = year)

# Preview the wide data
head(wideData, 5)
```

Now, let's convert this wide data back to a long format.

```
# Convert the wide format back to long format
longData <- wideData |>
  pivot_longer(!year, names_to = "song",
               values_to = "popularity")
```

```r
# Preview the long data
head(longData, 5)
```

For more information on ways to specify columns you want to select see the following help documentation.

```r
?tidyr_tidy_select
```

Handling Row Content Split and Reverse

Sometimes, data stored in a single column needs to be split into multiple columns or vice versa.

Merging Two Columns into One

Let's take the `genre` and `year` columns and merge them into a single column.

```r
# Combine 'genre' and 'year' into a single column
mergedData <- joinedData |>
  unite("genre_year", genre, year, sep = "_")

# Preview the merged dataset
head(mergedData, 5)
```

Splitting the Merged Column Back into Two

```r
# Split the 'genre_year' column back into 'genre' and 'year'
splitData <- mergedData |>
  separate(genre_year, into = c("genre", "year"), sep = "_")

# Preview the split dataset
head(splitData, 5)
```

Creating a Pseudo-Publication Ready Visualization

We'll combine all the data wrangling techniques you've learned—`filtering`, `selecting`, `mutating`, `summarizing`, and `joining`—to perform a detailed analysis and produce a polished, publication-ready visualization.

Creating a Professional-Quality Visualization

Here's a step-by-step guide to transform and visualize data from 10 countries in the dataset:

```
# Step 1: Join with the additional dataset to get more artist information
joinedDataV <- left_join(spotify, artists, by = "artist")

# Step 2: Mutate to get the dates in the correct format
joinedDataV <- joinedDataV |>
  mutate(date = ymd(release_date)) |>
  mutate(year = year(date)) |>
  mutate(month = month(date))

# Step 2: Filter the dataset to keep only Rap and R&B Songs from after 2010
filteredDataV <- joinedDataV |>
  filter(year >= 2010 &
           genre %in% c("R&b", "Rap"))

# Step 3: Select relevant columns for analysis
selectedDataV <- filteredDataV |>
  select(song, artist, genre, popularity, year, valence, danceability, avg_danceability)

# Step 4: Mutate the data to create new metric
mutatedDataV <- selectedDataV |>
  mutate(
    rel_danceability = danceability / avg_danceability)

# Step 5: Summarize the data to get average popularity per artist and average danceability (
summaryDataV <- mutatedDataV |>
  group_by(artist) |>
  summarize(
    avg_popularity = mean(popularity, na.rm = TRUE),
    avg_valence = mean(valence, na.rm = TRUE),
    time_avg_danceability = mean(danceability, na.rm = TRUE)
  ) |>
  ungroup() |>
  arrange(desc(avg_popularity)) |>
  head(10)

# Step 6: Create a publication-ready visualization
ggplot(summaryDataV, aes(
  x = time_avg_danceability,
  y = avg_valence,
```

```
  size = avg_popularity,  # Use average popularity for size mapping
  color = artist
)) +
  geom_point(alpha = 0.4) +
  scale_y_continuous(limits = c(0, 1)) +  # Adjust y-axis scale to go 0-1
  scale_x_continuous(limits = c(0.2, .90)) +  # Adjust x-axis scale to go 0.2 - .9 (better fi
  scale_size_continuous(
    range = c(5, 25),  # Adjust the bubble size range
    breaks = c(0,1)
  ) +
  scale_color_viridis_d(guide = "none") +  # Hide the color legend
  labs(
    title = "Relationship Between Danceability and Valence for Top 10 R&B/Rap Artists since 2
    subtitle = "Bubble size represents average popularity of songs per artist",
    x = "Average Danceability",
    y = "Average Valence (Song Positivity)"
  ) +
  geom_text(aes(label = artist), hjust = 0.5, size = 3, color = "black") + # Labels centered
  theme_minimal() +
  theme(
    text = element_text(family = "Arial"),
    plot.title = element_text(size = 11, face = "bold"),
    plot.subtitle = element_text(size = 9, face = "italic"),
    axis.title = element_text(size = 8),
    legend.title = element_text(size = 8),
    legend.text = element_text(size = 8),
    legend.position = "bottom"
  )
```

Review & Scrutinize

Answer the following two questions about the plot created above.

What is this visualization?: Write a brief paragraph describing the design, purpose, and key message of the plot. Explain what the visualization is intended to show and how it effectively communicates the data.

Critique: In 2-4 sentences, discuss what makes the plot polished and professional. Identify any characteristics of the plot that could be improved upon.

Submit your work

To demonstrate your understanding and comfort with wrangling data, you'll need to submit your completed and rendered document to Canvas.

Please ensure you have executable code in all of the chunks labeled `# Your code goes here...` and have written answers to the two questions stated in the *Review & Scrutinize* section.

In the YAML of this file, change `format` to `pdf` and `eval` to `true`. Then render the document. Submit both your .qmd and PDF files to the **Data Wrangling Lab Activity** on Canvas.